PROJECT 05 | CAPSTONE ASSIGNMENT

AI Data Engineering & Analytics Copilot

Industry: Data Engineering / BI | Focus: Natural-language analytics, data quality and supervised ETL

Project type: Enterprise capstone / production-style prototype

UI requirement: Streamlit ONLY: all application pages, dashboards and forms

Backend: FastAPI APIs + PostgreSQL (or equivalent relational DB)

01 Business challenge and objective

Data teams struggle with schema discovery, profiling, recurring ETL failures and natural-language analytics requests. Build a metadata-aware copilot connected to a realistic data pipeline.

02 Functional requirements

- Connect to PostgreSQL/MySQL and CSV/Parquet sources through configuration.
- Maintain searchable schema catalog, glossary, dataset owner and column descriptions.
- Ingest raw records to Bronze, clean/deduplicate to Silver and aggregate to Gold using PySpark.
- Collect null, uniqueness, data-type, referential and drift quality checks with alerts.
- Generate read-only SQL for authorized business questions with AST/allowlist and cost-limit validation.
- Execute only approved SELECT statements using read-only database credentials.
- Explain transformations and propose fixes; require explicit engineer approval before pipeline changes.
- Provide Streamlit dashboards for query results, lineage, pipeline status, errors and SLA metrics.
- Run scheduled ETL with retry and failure history and generate reports from Gold tables.

03 Agent and service design

- Metadata Agent: define responsibilities, inputs, outputs, approved tools, errors and hand-offs.
- SQL Planner: define responsibilities, inputs, outputs, approved tools, errors and hand-offs.
- Data Quality Agent: define responsibilities, inputs, outputs, approved tools, errors and hand-offs.
- ETL Recommendation Agent: define responsibilities, inputs, outputs, approved tools, errors and hand-offs.
- Implement graph flow: request -> classification/planning -> approved retrieval or tools -> verification -> response/action -> audit.
- Document which steps use deterministic code, traditional ML and LLM reasoning.

04 Streamlit application pages

- Login and role-specific navigation
- Business records and search/filter pages
- Main AI/agent workflow interface with evidence and status
- Human review/approval queue for consequential actions
- Admin/settings page and usage/monitoring dashboard
- Design clear validation messages, loading/progress states, file previews and download/export actions.
- Use Streamlit session state for UI continuity; store long-lived business and agent state in the backend database.

05 Data and integration requirements

- 100,000+ records across 3+ connected datasets, 2 complete ETL pipelines

- Datasets, schemas, lineage nodes, checks, pipeline runs, query audits, approval logs
- Implement at least two functioning integrations (real sandbox APIs or documented mock services).
- Provide synthetic seed scripts, realistic edge cases and instructions to reset test environments.

06 Suggested technology stack

- UI: Streamlit multipage application using forms, tables, filters, charts and authentication flow.
- Backend: Python, FastAPI, Pydantic, SQLAlchemy, Alembic, PostgreSQL.
- AI: model API or local Ollama; LangChain/LangGraph where justified; vector store for RAG projects.
- Background processing: job queue or equivalent worker for long-running document/agent tasks.
- Infra: Docker, GitHub Actions CI, optional AWS/Azure deployment; tracing via LangSmith or Langfuse.
- Avoid React, Angular, Vue, separate JavaScript frontend or frontend frameworks.

07 Non-functional and production controls

- Authentication with role-based authorization on Streamlit pages AND FastAPI endpoints
- FastAPI OpenAPI docs; input/output validation via Pydantic; database migrations
- Deterministic rules for calculations and validations; agents cannot bypass those rules
- LangGraph persisted state, conditional routing, bounded retries, timeouts and safe tool allowlists
- Redaction/PII handling, prompt-injection defenses, retrieval authorization and secrets via environment variables
- Structured application logs, LLM/tool traces, cost and latency metrics, error dashboards
- Unit tests, API/integration tests, agent tool tests, failure cases and an evaluation dataset
- Docker Compose for local run, production-oriented environment configuration and cloud deployment
- Audit trail for any approval and state-changing operation, including actor, time and tool result

08 User acceptance scenarios

- 20 test natural-language questions evaluated for SQL correctness
- DELETE/UPDATE/DROP and unauthorized tables blocked
- Lineage maps raw data to Gold reports
- A failed pipeline can be retried and diagnosed
- Demonstrate invalid input, simulated API outage, LLM timeout and retry/fallback behavior.
- Measure p50/p95 workflow latency, error rate, token usage and average cost per task on a defined test set.

09 Mandatory student deliverables

- Requirements (BRD/SRS), user stories and acceptance matrix
- System architecture diagram, agent graph diagram, ERD and workflow sequence diagram
- Repository with Streamlit application, FastAPI backend, agents, migrations, test suite and README
- Synthetic seed datasets and evaluation fixtures; no real confidential customer documents
- OpenAPI specifications, request examples and Postman collection
- Docker files, environment variable sample and deployment instructions
- Evaluation report with test cases, measured quality, limitations and risk mitigations
- 10-15 minute product demonstration with one success path and one failure/approval path

11 Evaluation rubric (100 marks)

Architecture and requirements 15 Database, FastAPI and integrations 15 GenAI/ML/RAG correctness 15 Agents, tools, state and approvals 20 Security, testing and reliability 15 Deployment and observability 10 Documentation and demo 10

11 Final review and completion gate

Minimum recommended score: 70/100; distinction: 85/100. Critical authorization, human-approval and data-integrity tests must pass regardless of total score. A polished UI alone does not constitute a production-ready project.